

# EEEC174AY Lab 1: Environment Setup

## 1 Overview

In this lab, you will set up the software workflow used for programming assignments in this course. You will:

1. Create or configure a GitHub repository for your course work.
2. Clone that repository onto either your own computer or the course server.
3. Launch the provided Docker development environment.
4. Connect to Jupyter Notebook and complete the programming assignment.
5. Submit the required files through Canvas.

**Important:** Canvas is the official system for assignment distribution, submission, deadlines, and grading. You must submit the required files to Canvas by the posted deadline, even if you also maintain your work in GitHub.

## 2 Software and Accounts

Before beginning, make sure you have:

- A GitHub account: <https://github.com/>
- Access to Canvas for this course
- Git installed on the machine where you will work
- Access to the course server and an assigned port number, if you will use the course server

You may work either on your own machine or on a course server. If you use a course server, you will need to connect through the UC Davis Library VPN or College of Engineering VPN. See <https://library.ucdavis.edu/vpn/> or [COE VPN](#) for installation and connection instructions.

## 3 Set Up Your GitHub Repository

You are expected to host your course projects in a GitHub repository. GitHub provides version control and a backup history for your work. It is separate from Canvas: GitHub is where you develop and track your project, while Canvas is where you submit work for grading. Keep your Github repository private, but be expected to share it with others for evaluation.

### 3.1 Create a Repository on GitHub

1. Sign in to GitHub at <https://github.com/>.
2. Click the + menu in the upper-right corner and choose **New repository**.
3. Choose a repository name. Use a clear name such as **EEC174-Lab1**.
4. Select **Private** unless your instructor explicitly tells you to use a public repository.
5. Check **Add a README file**.
6. Find **Add a .gitignore file** and **Select Python Template**.
7. Click **Create repository**.

**Important:** Do not place passwords, API keys, private datasets, personally identifiable information, or other sensitive material in a GitHub repository. A private repository limits access, but it is still not an appropriate location for secrets.

### 3.2 Clone the Repository

You can clone the repository on your personal machine and/or the course server. In the GitHub repository page, click the green **Code** button, select **HTTPS**, and copy the repository URL.

In the terminal on the machine where you will work, run:

```
git clone <your-https-repository-url>
cd <your-repository-directory>
```

For example:

```
git clone https://github.com/your-github-username/eec174ay-lab1.git
cd eec174ay-lab1
```

If GitHub asks for authentication when you push changes, follow GitHub's instructions for HTTPS authentication. GitHub does not accept an account password for Git operations over HTTPS; use the sign-in method or personal access token supported by your Git setup.

### 3.3 Add Lab Files to Your Repository

Download the Lab 1 files from Canvas. Copy the provided files into an appropriate folder in your repository. A suggested project layout is:

```
eec174ay-labs/
- README.md
- Report.md
- Lab1.ipynb
- tutorials/
  -- Intro2Python.ipynb
```

Commit the starter files before you begin editing them:

```
git add *
git commit -m "Add Lab 1 starter files"
git push -u origin main
```

If your repository uses a branch name other than `main`, replace `main` with your repository's default branch name.

## 4 Working on a Course Server

This section applies only if you are completing the lab on a course server. You will be assigned a server (for example, Hades, Atlas, or Kronos) and an available port number. The examples below use the server `atlas.ece.ucdavis.edu` and port 8080; replace both values with the server and port assigned to you.

### 4.1 Connect with SSH Port Forwarding

On your *own computer*, open a terminal and run:

```
ssh -L <assigned_port>:localhost:<assigned_port> \  
<kerberos_username>@<server_hostname>
```

For example:

```
ssh -L 8080:localhost:8080 yuiishi@atlas.ece.ucdavis.edu
```

Keep this SSH terminal open while you use Jupyter. This command creates an encrypted tunnel from your laptop's `localhost:<assigned_port>` to the corresponding port on the course server.

## 5 Launching the Docker Environment

The course server provides a Docker image containing the software needed for this lab. A Docker *image* is the reusable packaged environment; a Docker *container* is a running instance of that image. First, make sure you are in the root of your cloned repository:

```
cd <your-repository-directory>
```

Launch the container. Replace `<assigned_port>` and `<kerberos_ID>` with your assigned values:

```
docker run -it \  
  --mount src="$(pwd)",target=/workspace,type=bind \  
  -p 127.0.0.1:<assigned_port>:<assigned_port> \  
  --name <kerberos_ID>_lab1 \  
  eec174lab1-2:latest
```

This command:

- Uses `-it` to start an interactive terminal inside the container.
- Uses `--mount` to make the repository directory available inside the container at `/workspace`. Files saved in `/workspace` are saved in your repository on the server.
- Uses `-p` to map your assigned server port to the same port inside the container. Binding to `127.0.0.1` limits access to the server itself, so you connect through the SSH tunnel rather than exposing Jupyter directly to the network.

- Uses `-name` to give the container a predictable name.

After the container starts, you are already in the mounted clone at `/workspace`.

## 5.1 Restarting or Removing a Container

If you exit a container, it stops but remains available under the name you assigned. To resume it later, run this command on the course server:

```
docker start -ai <kerberos_ID>_lab1
```

If you need to delete the stopped container and create a fresh one, run:

```
docker rm <kerberos_ID>_lab1
```

Then run the `docker run` command again. Your work in `/workspace` remains in your Git repository on the server because it is stored through the bind mount.

## 6 Launching and Connecting to Jupyter Notebook

Inside the Docker container, after confirming that you are in `/workspace`, launch Jupyter Notebook with:

```
jupyter notebook --no-browser --ip=0.0.0.0 \  
--allow-root --port=<assigned_port>
```

For the example port 8080:

```
jupyter notebook --no-browser --ip=0.0.0.0 \  
--allow-root --port=8080
```

Jupyter will print one or more URLs containing a security token. On your own computer, open the printed URL in a browser, replacing the host with `localhost` if necessary. For example:

```
http://localhost:8080/tree?token=<token_from_terminal>
```

You should see the files in your cloned repository. Open `Lab1.ipynb` and complete the programming exercises.

When you are finished with Jupyter, press `Ctrl+C` in the container terminal to stop the notebook server. Type `exit` to leave the container, and type `exit` again in the SSH terminal when you are done with the remote server.

## 7 Assignment

Complete all Python questions in `Lab1.ipynb`. The tutorial notebook `tutorials/Intro2Python.ipynb` may be helpful.

Before all future submissions for this course:

1. Restart the notebook kernel and run all cells from top to bottom.
2. Resolve all errors.

3. Confirm that required outputs, figures, and written responses are visible.
4. Save the notebook.
5. Commit and push your work to GitHub.
6. Submit the required file(s) through Canvas before the deadline.

## 8 Report

Submit a written report answering the following questions. Unless Canvas specifies a different format, write your report in Markdown as `report.md` and keep a copy in this lab's GitHub repository.

### 1. SSH

- (a) What is the purpose of port forwarding between your laptop, the course server, and the Docker container?

### 2. Git and GitHub

- (a) What is a Git repository? Explain specifically why you would track your code in a repository.
- (b) Explain what a branch is in Git. Why are branches useful when a team works on one project?
- (c) What is the default branch (commonly named `main`)? What is its role in a software project?
- (d) Explain what a pull request is and why it is important. Briefly explain a typical collaborative software-development cycle using Git and GitHub.

### 3. Docker

- (a) In your own words, explain why Docker is useful for programmers. Discuss the ways Docker can save development time.
- (b) Explain the difference between a Docker image and a Docker container.
- (c) Explain how Docker can resolve a dependency conflict between a computer-vision program and a control program being developed on the same machine.
- (d) Can Docker be used to support Linux software on a Windows computer? Explain the role of the Linux environment used by Docker Desktop.
- (e) Can Docker support development of the same project on different machines? Explain.
- (f) In one to three sentences, explain how to build a new Docker image from scratch.

### 4. Python

- (a) Compare interpreted and compiled languages. In what situations might Python be more advantageous than C++?
- (b) Explain what lists, dictionaries, and tuples are in Python. Give one example scenario where you would use each.

## 9 Submission Instructions

### 9.1 Programming Assignment

Complete all programming tasks in `Lab1.ipynb`. Submit the completed notebook to the Lab 1 Canvas assignment by the posted deadline.

Canvas records the official submission timestamp. Your GitHub repository is expected to contain your work and version history, but it does not replace the Canvas submission.

### 9.2 Written Report

Submit your written report to the Lab 1 Canvas assignment in the format specified on Canvas. Keep a copy of the report in this lab's GitHub repository, preferably as `report.md` in the repository root.